you call `join()`. This means that the call to `join()` is liable to be skipped if an exception is thrown after the thread has been started but before the call to `join()`.

To avoid your application being terminated when an exception is thrown, you therefore need to make a decision on what to do in this case. In general, if you were intending to call `join()` in the non-exceptional case, you also need to call `join()` in the presence of an exception to avoid accidental lifetime problems. The next listing shows some simple code that does just that.

Listing 2.2. Waiting for a thread to finish

```
struct func;                                    <- See definition in listing 2.1

void f()
{
    int some_local_state=0;
    func my_func(some_local_state);
    std::thread t(my_func);
    try
    {
        do_something_in_current_thread();
    }
    catch(...)
    {
        t.join();                               <- (1)
        throw;
    }
    t.join();                                   <- (2)
}
```

The code in listing 2.2 uses a `try`/`catch` block to ensure that a thread with access to local state is finished before the function exits, whether the function exits normally (2) or by an exception (1). The use of `try`/`catch` blocks is verbose, and it's easy to get the scope slightly wrong, so this isn't an ideal scenario. If it's important to ensure that the thread must complete before the function exits—whether because it has a reference to other local variables or for any other reason—then it's important to ensure this is the case for all possible exit paths, whether normal or exceptional, and it's desirable to provide a simple, concise mechanism for doing so.